Capstone Project — Predicting 30-Day Hospital Readmission Risk

Problem Statement

This notebook builds an end-to-end machine learning pipeline to predict whether a patient will be **readmitted within 30 days** of discharge.

Target transformation

The original `readmitted` column contains:

- `<30`
- `>30`
- `NO`

It is transformed into a binary target:

- `1` = readmitted within 30 days
- `0` = not readmitted within 30 days

Capstone alignment

This notebook is organized in three phases:

Phase 1 — Data Processing

- data cleaning
- handling missing values
- exploratory data analysis (EDA)
- categorical encoding
- feature scaling
- stratified train/test split

Phase 2 — Machine Learning

- compare multiple models:
  - Logistic Regression
  - Random Forest
  - Gradient Boosting
  - Neural Network (MLPClassifier)
- 5-fold cross-validation
- precision / recall / F1 / ROC-AUC
- hyperparameter tuning with `RandomizedSearchCV`

Phase 3 — Generative AI

- build a simple patient-level prompt grounded in model outputs
- generate a structured clinical-style summary
- document governance limits: GenAI is **not** used to make predictions

1) Imports and configuration

```
import os
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import (
    train_test_split,
    StratifiedKFold,
    cross_validate,
    RandomizedSearchCV
)
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder, StandardScaler

from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.neural_network import MLPClassifier

from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    ConfusionMatrixDisplay,
    roc_auc_score,
    RocCurveDisplay,
    precision_score,
    recall_score,
    f1_score,
    accuracy_score
)

try:
    import joblib
except ImportError:
    joblib = None

RANDOM_STATE = 42
TEST_SIZE = 0.20
CV_FOLDS = 5

np.random.seed(RANDOM_STATE)
warnings.filterwarnings("ignore")

pd.set_option("display.max_columns", 200)
pd.set_option("display.max_rows", 100)
```

2) Load the public dataset

- Expected local file path: `data/diabetic_data.csv`
- If the file is not found, download the **Diabetes 130-US hospitals for years 1999–2008** dataset and place `diabetic_data.csv` inside the `data/` folder.

```
DATA_PATH = os.path.join("data", "diabetic_data.csv")

if not os.path.exists(DATA_PATH):
    raise FileNotFoundError(
        f"Dataset not found at: {DATA_PATH}\n"
        "Place 'diabetic_data.csv' inside the data/ folder before running the notebook."
    )

df_raw = pd.read_csv(DATA_PATH)
print("Dataset loaded successfully.")
print("Shape:", df_raw.shape)
display(df_raw.head())
```

```
Dataset loaded successfully.
Shape: (101766, 50)
```

	encounter_id	patient_nbr	race	gender	age	weight	admission_type_id	discharge_disposition_id	admission_source_id	time_in_hospital	payer_code	medical_specialty	num_lab_procedures	num_procedures	num_medications	number_outpatient	number_emergency	number_inpatient	diag_1	diag_2	diag_3	number_diagnoses	max_glu_serum	A1Cresult	metformin	repaglinide	nateglini
0	2278392	8222157	Caucasian	Female	[0-10)	?	6	25	1	1	?	Pediatrics-Endocrinology	41	0	1	0	0	0	250.83	?	?	1	NaN	NaN	No	No	N
1	149190	55629189	Caucasian	Female	[10-20)	?	1	1	7	3	?	?	59	0	18	0	0	0	276	250.01	255	9	NaN	NaN	No	No	N
2	64410	86047875	AfricanAmerican	Female	[20-30)	?	1	1	7	2	?	?	11	5	13	2	0	1	648	250	V27	6	NaN	NaN	No	No	N
3	500364	82442376	Caucasian	Male	[30-40)	?	1	1	7	2	?	?	44	1	16	0	0	0	8	250.43	403	7	NaN	NaN	No	No	N
4	16680	42519267	Caucasian	Male	[40-50)	?	1	1	7	1	?	?	51	0	8	0	0	0	197	157	250	5	NaN	NaN	No	No	N

3) Data Quality Audit

In this section, we perform an initial data audit and exploratory analysis.

We examine :

• dataset structure and dimensions
• data types
• missing values (NaN and hidden '?')
• duplicated rows
• feature cardinality
• numerical feature statistics
• categorical feature distributions
This step helps identify potential data quality issues before data cleaning and preprocessing.

```
# Work on a copy
df = df_raw.copy()

# ========================
# 1. DATASET OVERVIEW
# ========================
print("=" * 70)
print("1. DATASET OVERVIEW")
print("=" * 70)
print(f"Rows: {df.shape[0]}")
print(f"Columns: {df.shape[1]}")
display(df.head())

# ========================
# 2. DATA TYPES
# ========================
print("\n" + "=" * 70)
print("2. DATA TYPES")
print("=" * 70)
display(df.dtypes.to_frame("dtype"))

# ========================
# 3. GLOBAL DATA QUALITY SUMMARY
# ========================
print("\n" + "=" * 70)
print("3. GLOBAL DATA QUALITY SUMMARY")
print("=" * 70)

summary = pd.DataFrame({
    "dtype": df.dtypes.astype(str),
    "non_null_count": df.notnull().sum(),
    "missing_nan_count": df.isnull().sum(),
    "missing_nan_percent": ((df.isnull().sum() / len(df)) * 100).round(2),
    "unique_values": df.nunique(dropna=True)
})

display(summary.sort_values(by="missing_nan_count", ascending=False))

# ========================
# 4. MISSING VALUES (NaN)
# ========================
print("\n" + "=" * 70)
print("4. MISSING VALUES (NaN)")
print("=" * 70)

missing_nan = df.isnull().sum().to_frame("missing_nan_count")
missing_nan["missing_nan_percent"] = ((missing_nan["missing_nan_count"] / len(df)) * 100).round(2)

display(missing_nan.sort_values(by="missing_nan_count", ascending=False))

# ========================
# 5. HIDDEN MISSING VALUES CODED AS '?'
# ========================
print("\n" + "=" * 70)
print("5. HIDDEN MISSING VALUES CODED AS '?'")
```

```
print("=" * 70)

question_marks = (df.astype(str) == "?").sum().to_frame("question_mark_count")
question_marks["question_mark_percent"] = ((question_marks["question_mark_count"] / len(df)) * 100).round(2)

display(question_marks.sort_values(by="question_mark_count", ascending=False))

# ========================
# 6. COMBINED MISSING INDICATORS
# ========================
print("\n" + "=" * 70)
print("6. COMBINED MISSING INDICATORS (NaN + '?')")
print("=" * 70)

combined_missing = summary.copy()
combined_missing["question_mark_count"] = (df.astype(str) == "?").sum()
combined_missing["question_mark_percent"] = ((combined_missing["question_mark_count"] / len(df)) * 100).round(2)
combined_missing["total_missing_like_count"] = (
    combined_missing["missing_nan_count"] + combined_missing["question_mark_count"]
)
combined_missing["total_missing_like_percent"] = (
    (combined_missing["total_missing_like_count"] / len(df)) * 100
).round(2)

display(combined_missing.sort_values(by="total_missing_like_count", ascending=False))

# ========================
# 7. FEATURE CARDINALITY
# ========================
print("\n" + "=" * 70)
print("7. FEATURE CARDINALITY (NUMBER OF UNIQUE VALUES)")
print("=" * 70)

cardinality = df.nunique(dropna=True).to_frame("unique_values")
display(cardinality.sort_values(by="unique_values", ascending=True))

print("\nColumns with only 1 unique value:")
one_value_cols = cardinality[cardinality["unique_values"] <= 1]
display(one_value_cols)

# ========================
# 8. DUPLICATED ROWS
# ========================
print("\n" + "=" * 70)
print("8. DUPLICATED ROWS")
print("=" * 70)

duplicate_count = df.duplicated().sum()
duplicate_percent = round((duplicate_count / len(df)) * 100, 2)

print(f"Duplicated rows: {duplicate_count}")
print(f"Duplicated rows percent: {duplicate_percent}%")

# ========================
# 9. NUMERICAL VARIABLES SUMMARY
# ========================
print("\n" + "=" * 70)
print("9. NUMERICAL VARIABLES SUMMARY")
print("=" * 70)

num_df = df.select_dtypes(include=[np.number])

if not num_df.empty:
    numeric_summary = num_df.describe().T
    numeric_summary["missing_count"] = num_df.isnull().sum()
    numeric_summary["missing_percent"] = ((num_df.isnull().sum() / len(df)) * 100).round(2)
    display(numeric_summary)
else:
    print("No numerical columns found.")

# ========================
# 10. CATEGORICAL VARIABLES SUMMARY
# ========================
print("\n" + "=" * 70)
print("10. CATEGORICAL VARIABLES SUMMARY")
print("=" * 70)

cat_df = df.select_dtypes(include=["object", "category"])

if not cat_df.empty:
    categorical_summary = cat_df.describe().T
    categorical_summary["missing_nan_count"] = cat_df.isnull().sum()
    categorical_summary["question_mark_count"] = (cat_df.astype(str) == "?").sum()
    display(categorical_summary)
else:
    print("No categorical columns found.")

# ========================
# 11. TOP CATEGORIES PER CATEGORICAL COLUMN
# ========================
print("\n" + "=" * 70)
print("11. TOP CATEGORIES PER CATEGORICAL COLUMN")
print("=" * 70)

for col in cat_df.columns:
    print("\n" + "-" * 50)
    print(f"Column: {col}")
    print("-" * 50)
    display(df[col].value_counts(dropna=False).head(10).to_frame("count"))

# ========================
# 12. QUICK DATA QUALITY FLAGS
# ========================
print("\n" + "=" * 70)
print("12. QUICK DATA QUALITY FLAGS")
print("=" * 70)

flags = pd.DataFrame(index=df.columns)
flags["has_nan"] = df.isnull().sum() > 0
flags["has_question_mark"] = (df.astype(str) == "?").sum() > 0
flags["is_constant"] = df.nunique(dropna=True) <= 1
flags["high_cardinality"] = df.nunique(dropna=True) > 50

display(flags)
```

```
======================================================================
1. DATASET OVERVIEW
======================================================================
Rows: 101766
Columns: 50
```

	encounter_id	patient_nbr	race	gender	age	weight	admission_type_id	discharge_disposition_id	admission_source_id	time_in_hospital	payer_code	medical_specialty	num_lab_procedures	num_procedures	num_medications	number_outpatient	number_emergency	number_inpatient	diag_1	diag_2	diag_3	number_diagnoses	max_glu_serum	A1Cresult	metformin	repaglinide	nateglini
0	2278392	8222157	Caucasian	Female	[0-10)	?	6	25	1	1	?	Pediatrics-Endocrinology	41	0	1	0	0	0	250.83	?	?	1	NaN	NaN	No	No	N
1	149190	55629189	Caucasian	Female	[10-20)	?	1	1	7	3	?	?	59	0	18	0	0	0	276	250.01	255	9	NaN	NaN	No	No	N
2	64410	86047875	AfricanAmerican	Female	[20-30)	?	1	1	7	2	?	?	11	5	13	2	0	1	648	250	V27	6	NaN	NaN	No	No	N
3	500364	82442376	Caucasian	Male	[30-40)	?	1	1	7	2	?	?	44	1	16	0	0	0	8	250.43	403	7	NaN	NaN	No	No	N
4	16680	42519267	Caucasian	Male	[40-50)	?	1	1	7	1	?	?	51	0	8	0	0	0	197	157	250	5	NaN	NaN	No	No	N

```
======================================================================
2. DATA TYPES
======================================================================
```

	dtype
encounter_id	int64
patient_nbr	int64
race	object
gender	object
age	object
weight	object
admission_type_id	int64
discharge_disposition_id	int64
admission_source_id	int64
time_in_hospital	int64
payer_code	object
medical_specialty	object
num_lab_procedures	int64
num_procedures	int64
num_medications	int64
number_outpatient	int64
number_emergency	int64
number_inpatient	int64
diag_1	object
diag_2	object
diag_3	object
number_diagnoses	int64
max_glu_serum	object
A1Cresult	object
metformin	object
repaglinide	object
nateglinide	object
chlorpropamide	object
glimepiride	object
acetohexamide	object
glipizide	object
glyburide	object
tolbutamide	object
pioglitazone	object
rosiglitazone	object
acarbose	object
miglitol	object
troglitazone	object
tolazamide	object
examide	object
citoglipton	object
insulin	object
glyburide-metformin	object
glipizide-metformin	object
glimepiride-pioglitazone	object
metformin-rosiglitazone	object
metformin-pioglitazone	object
change	object
diabetesMed	object
readmitted	object

```
======================================================================
3. GLOBAL DATA QUALITY SUMMARY
======================================================================
```

	dtype	non_null_count	missing_nan_count	missing_nan_percent	unique_values
max_glu_serum	object	5346	96420	94.75	3
A1Cresult	object	17018	84748	83.28	3
race	object	101766	0	0.00	6
gender	object	101766	0	0.00	3
age	object	101766	0	0.00	10
weight	object	101766	0	0.00	10
admission_type_id	int64	101766	0	0.00	8
discharge_disposition_id	int64	101766	0	0.00	26
admission_source_id	int64	101766	0	0.00	17
time_in_hospital	int64	101766	0	0.00	14
payer_code	object	101766	0	0.00	18
medical_specialty	object	101766	0	0.00	73
num_lab_procedures	int64	101766	0	0.00	118
num_procedures	int64	101766	0	0.00	7
num_medications	int64	101766	0	0.00	75
number_outpatient	int64	101766	0	0.00	39
encounter_id	int64	101766	0	0.00	101766
patient_nbr	int64	101766	0	0.00	71518
number_inpatient	int64	101766	0	0.00	21
number_emergency	int64	101766	0	0.00	33
diag_1	object	101766	0	0.00	717
diag_2	object	101766	0	0.00	749
number_diagnoses	int64	101766	0	0.00	16
diag_3	object	101766	0	0.00	790
metformin	object	101766	0	0.00	4
repaglinide	object	101766	0	0.00	4
nateglinide	object	101766	0	0.00	4
chlorpropamide	object	101766	0	0.00	4
glimepiride	object	101766	0	0.00	4
acetohexamide	object	101766	0	0.00	2
glipizide	object	101766	0	0.00	4
glyburide	object	101766	0	0.00	4
tolbutamide	object	101766	0	0.00	2
pioglitazone	object	101766	0	0.00	4
rosiglitazone	object	101766	0	0.00	4
acarbose	object	101766	0	0.00	4
miglitol	object	101766	0	0.00	4
troglitazone	object	101766	0	0.00	2
tolazamide	object	101766	0	0.00	3
examide	object	101766	0	0.00	1
citoglipton	object	101766	0	0.00	1
insulin	object	101766	0	0.00	4
glyburide-metformin	object	101766	0	0.00	4
glipizide-metformin	object	101766	0	0.00	2
glimepiride-pioglitazone	object	101766	0	0.00	2
metformin-rosiglitazone	object	101766	0	0.00	2
metformin-pioglitazone	object	101766	0	0.00	2
change	object	101766	0	0.00	2
diabetesMed	object	101766	0	0.00	2
readmitted	object	101766	0	0.00	3

```
======================================================================
4. MISSING VALUES (NaN)
======================================================================
```

	missing_nan_count	missing_nan_percent
max_glu_serum	96420	94.75
A1Cresult	84748	83.28
race	0	0.00
gender	0	0.00
age	0	0.00
weight	0	0.00
admission_type_id	0	0.00
discharge_disposition_id	0	0.00
admission_source_id	0	0.00
time_in_hospital	0	0.00
payer_code	0	0.00
medical_specialty	0	0.00
num_lab_procedures	0	0.00
num_procedures	0	0.00
num_medications	0	0.00
number_outpatient	0	0.00
encounter_id	0	0.00
patient_nbr	0	0.00
number_inpatient	0	0.00

number_emergency	0	0.00
diag_1	0	0.00
diag_2	0	0.00
number_diagnoses	0	0.00
diag_3	0	0.00
metformin	0	0.00
repaglinide	0	0.00
nateglinide	0	0.00
chlorpropamide	0	0.00
glimepiride	0	0.00
acetohexamide	0	0.00
glipizide	0	0.00
glyburide	0	0.00
tolbutamide	0	0.00
pioglitazone	0	0.00
rosiglitazone	0	0.00
acarbose	0	0.00
miglitol	0	0.00
troglitazone	0	0.00
tolazamide	0	0.00
examide	0	0.00
citoglipton	0	0.00
insulin	0	0.00
glyburide-metformin	0	0.00
glipizide-metformin	0	0.00
glimepiride-pioglitazone	0	0.00
metformin-rosiglitazone	0	0.00
metformin-pioglitazone	0	0.00
change	0	0.00
diabetesMed	0	0.00
readmitted	0	0.00

```
====================================================================
5. HIDDEN MISSING VALUES CODED AS '?'
====================================================================
```

	question_mark_count	question_mark_percent
weight	98569	96.86
medical_specialty	49949	49.08
payer_code	40256	39.56
race	2273	2.23
diag_3	1423	1.40
diag_2	358	0.35
diag_1	21	0.02
admission_type_id	0	0.00
patient_nbr	0	0.00
encounter_id	0	0.00
time_in_hospital	0	0.00
admission_source_id	0	0.00
num_lab_procedures	0	0.00
num_procedures	0	0.00
num_medications	0	0.00
discharge_disposition_id	0	0.00
gender	0	0.00
age	0	0.00
number_inpatient	0	0.00
number_emergency	0	0.00
number_outpatient	0	0.00
number_diagnoses	0	0.00
max_glu_serum	0	0.00
A1Cresult	0	0.00
metformin	0	0.00
repaglinide	0	0.00
nateglinide	0	0.00
chlorpropamide	0	0.00
glimepiride	0	0.00
acetohexamide	0	0.00
glipizide	0	0.00
glyburide	0	0.00
tolbutamide	0	0.00
pioglitazone	0	0.00
rosiglitazone	0	0.00
acarbose	0	0.00
miglitol	0	0.00
troglitazone	0	0.00
tolazamide	0	0.00
examide	0	0.00
citoglipton	0	0.00
insulin	0	0.00
glyburide-metformin	0	0.00
glipizide-metformin	0	0.00
glimepiride-pioglitazone	0	0.00
metformin-rosiglitazone	0	0.00
metformin-pioglitazone	0	0.00
change	0	0.00
diabetesMed	0	0.00
readmitted	0	0.00

```
====================================================================
6. COMBINED MISSING INDICATORS (NaN + '?')
====================================================================
```

	dtype	non_null_count	missing_nan_count	missing_nan_percent	unique_values	question_mark_count	question_mark_percent	total_missing_like_count	total_missing_like_percent
weight	object	101766	0	0.00	10	98569	96.86	98569	96.86
max_glu_serum	object	5346	96420	94.75	3	0	0.00	96420	94.75
A1Cresult	object	17018	84748	83.28	3	0	0.00	84748	83.28
medical_specialty	object	101766	0	0.00	73	49949	49.08	49949	49.08
payer_code	object	101766	0	0.00	18	40256	39.56	40256	39.56
race	object	101766	0	0.00	6	2273	2.23	2273	2.23
diag_3	object	101766	0	0.00	790	1423	1.40	1423	1.40
diag_2	object	101766	0	0.00	749	358	0.35	358	0.35
diag_1	object	101766	0	0.00	717	21	0.02	21	0.02
patient_nbr	int64	101766	0	0.00	71518	0	0.00	0	0.00
time_in_hospital	int64	101766	0	0.00	14	0	0.00	0	0.00
admission_source_id	int64	101766	0	0.00	17	0	0.00	0	0.00
num_lab_procedures	int64	101766	0	0.00	118	0	0.00	0	0.00
encounter_id	int64	101766	0	0.00	101766	0	0.00	0	0.00
admission_type_id	int64	101766	0	0.00	8	0	0.00	0	0.00
discharge_disposition_id	int64	101766	0	0.00	26	0	0.00	0	0.00
gender	object	101766	0	0.00	3	0	0.00	0	0.00
age	object	101766	0	0.00	10	0	0.00	0	0.00
number_inpatient	int64	101766	0	0.00	21	0	0.00	0	0.00
number_emergency	int64	101766	0	0.00	33	0	0.00	0	0.00
number_outpatient	int64	101766	0	0.00	39	0	0.00	0	0.00
num_medications	int64	101766	0	0.00	75	0	0.00	0	0.00
num_procedures	int64	101766	0	0.00	7	0	0.00	0	0.00
number_diagnoses	int64	101766	0	0.00	16	0	0.00	0	0.00
metformin	object	101766	0	0.00	4	0	0.00	0	0.00
repaglinide	object	101766	0	0.00	4	0	0.00	0	0.00
nateglinide	object	101766	0	0.00	4	0	0.00	0	0.00
chlorpropamide	object	101766	0	0.00	4	0	0.00	0	0.00
glimepiride	object	101766	0	0.00	4	0	0.00	0	0.00
acetohexamide	object	101766	0	0.00	2	0	0.00	0	0.00
glipizide	object	101766	0	0.00	4	0	0.00	0	0.00
glyburide	object	101766	0	0.00	4	0	0.00	0	0.00
tolbutamide	object	101766	0	0.00	2	0	0.00	0	0.00
pioglitazone	object	101766	0	0.00	4	0	0.00	0	0.00
rosiglitazone	object	101766	0	0.00	4	0	0.00	0	0.00
acarbose	object	101766	0	0.00	4	0	0.00	0	0.00
miglitol	object	101766	0	0.00	4	0	0.00	0	0.00
troglitazone	object	101766	0	0.00	2	0	0.00	0	0.00
tolazamide	object	101766	0	0.00	3	0	0.00	0	0.00
examide	object	101766	0	0.00	1	0	0.00	0	0.00
citoglipton	object	101766	0	0.00	1	0	0.00	0	0.00
insulin	object	101766	0	0.00	4	0	0.00	0	0.00
glyburide-metformin	object	101766	0	0.00	4	0	0.00	0	0.00
glipizide-metformin	object	101766	0	0.00	2	0	0.00	0	0.00
glimepiride-pioglitazone	object	101766	0	0.00	2	0	0.00	0	0.00
metformin-rosiglitazone	object	101766	0	0.00	2	0	0.00	0	0.00
metformin-pioglitazone	object	101766	0	0.00	2	0	0.00	0	0.00
change	object	101766	0	0.00	2	0	0.00	0	0.00
diabetesMed	object	101766	0	0.00	2	0	0.00	0	0.00
readmitted	object	101766	0	0.00	3	0	0.00	0	0.00

```
====================================================================
7. FEATURE CARDINALITY (NUMBER OF UNIQUE VALUES)
====================================================================
```

	unique_values

citoglipton	1
examide	1
acetohexamide	2
glipizide-metformin	2
tolbutamide	2
troglitazone	2
metformin-rosiglitazone	2
glimepiride-pioglitazone	2
metformin-pioglitazone	2
change	2
diabetesMed	2
gender	3
tolazamide	3
readmitted	3
max_glu_serum	3
A1Cresult	3
acarbose	4
rosiglitazone	4
pioglitazone	4
miglitol	4
glimepiride	4
metformin	4
nateglinide	4
repaglinide	4
chlorpropamide	4
glipizide	4
glyburide	4
insulin	4
glyburide-metformin	4
race	6
num_procedures	7
admission_type_id	8
age	10
weight	10
time_in_hospital	14
number_diagnoses	16
admission_source_id	17
payer_code	18
number_inpatient	21
discharge_disposition_id	26
number_emergency	33
number_outpatient	39
medical_specialty	73
num_medications	75
num_lab_procedures	118
diag_1	717
diag_2	749
diag_3	790
patient_nbr	71518
encounter_id	101766

```
Columns with only 1 unique value:
```

	unique_values
examide	1
citoglipton	1

```
======================================================================
8. DUPLICATED ROWS
======================================================================
Duplicated rows: 0
Duplicated rows percent: 0.0%

======================================================================
9. NUMERICAL VARIABLES SUMMARY
======================================================================
```

	count	mean	std	min	25%	50%	75%	max	missing_count	missing_percent
encounter_id	101766.0	1.652016e+08	1.026403e+08	12522.0	84961194.0	152388987.0	2.302709e+08	443867222.0	0	0.0
patient_nbr	101766.0	5.433040e+07	3.869636e+07	135.0	23413221.0	45505143.0	8.754595e+07	189502619.0	0	0.0
admission_type_id	101766.0	2.024006e+00	1.445403e+00	1.0	1.0	1.0	3.000000e+00	8.0	0	0.0
discharge_disposition_id	101766.0	3.715642e+00	5.280166e+00	1.0	1.0	1.0	4.000000e+00	28.0	0	0.0
admission_source_id	101766.0	5.754437e+00	4.064081e+00	1.0	1.0	7.0	7.000000e+00	25.0	0	0.0
time_in_hospital	101766.0	4.395987e+00	2.985108e+00	1.0	2.0	4.0	6.000000e+00	14.0	0	0.0
num_lab_procedures	101766.0	4.309564e+01	1.967436e+01	1.0	31.0	44.0	5.700000e+01	132.0	0	0.0
num_procedures	101766.0	1.339730e+00	1.705807e+00	0.0	0.0	1.0	2.000000e+00	6.0	0	0.0
num_medications	101766.0	1.602184e+01	8.127566e+00	1.0	10.0	15.0	2.000000e+01	81.0	0	0.0
number_outpatient	101766.0	3.693572e-01	1.267265e+00	0.0	0.0	0.0	0.000000e+00	42.0	0	0.0
number_emergency	101766.0	1.978362e-01	9.304723e-01	0.0	0.0	0.0	0.000000e+00	76.0	0	0.0
number_inpatient	101766.0	6.355659e-01	1.262863e+00	0.0	0.0	0.0	1.000000e+00	21.0	0	0.0
number_diagnoses	101766.0	7.422607e+00	1.933600e+00	1.0	6.0	8.0	9.000000e+00	16.0	0	0.0

```
======================================================================
10. CATEGORICAL VARIABLES SUMMARY
======================================================================
```

	count	unique	top	freq	missing_nan_count	question_mark_count
race	101766	6	Caucasian	76099	0	2273
gender	101766	3	Female	54708	0	0
age	101766	10	[70-80)	26068	0	0
weight	101766	10	?	98569	0	98569
payer_code	101766	18	?	40256	0	40256
medical_specialty	101766	73	?	49949	0	49949
diag_1	101766	717	428	6862	0	21
diag_2	101766	749	276	6752	0	358
diag_3	101766	790	250	11555	0	1423
max_glu_serum	5346	3	Norm	2597	96420	0
A1Cresult	17018	3	>8	8216	84748	0
metformin	101766	4	No	81778	0	0
repaglinide	101766	4	No	100227	0	0
nateglinide	101766	4	No	101063	0	0
chlorpropamide	101766	4	No	101680	0	0
glimepiride	101766	4	No	96575	0	0
acetohexamide	101766	2	No	101765	0	0
glipizide	101766	4	No	89080	0	0
glyburide	101766	4	No	91116	0	0
tolbutamide	101766	2	No	101743	0	0
pioglitazone	101766	4	No	94438	0	0
rosiglitazone	101766	4	No	95401	0	0
acarbose	101766	4	No	101458	0	0
miglitol	101766	4	No	101728	0	0
troglitazone	101766	2	No	101763	0	0
tolazamide	101766	3	No	101727	0	0
examide	101766	1	No	101766	0	0
citoglipton	101766	1	No	101766	0	0
insulin	101766	4	No	47383	0	0
glyburide-metformin	101766	4	No	101060	0	0
glipizide-metformin	101766	2	No	101753	0	0
glimepiride-pioglitazone	101766	2	No	101765	0	0
metformin-rosiglitazone	101766	2	No	101764	0	0
metformin-pioglitazone	101766	2	No	101765	0	0
change	101766	2	No	54755	0	0
diabetesMed	101766	2	Yes	78363	0	0
readmitted	101766	3	NO	54864	0	0

```
======================================================================
11. TOP CATEGORIES PER CATEGORICAL COLUMN
======================================================================

--------------------------------------------------
Column: race
--------------------------------------------------
```

race	count
Caucasian	76099
AfricanAmerican	19210
?	2273
Hispanic	2037
Other	1506
Asian	641

```
--------------------------------------------------
Column: gender
--------------------------------------------------
```

gender	count
Female	54708
Male	47055
Unknown/Invalid	3

```
--------------------------------------------------
Column: age
--------------------------------------------------
```

age	count
[70-80)	26068
[60-70)	22483
[50-60)	17256

[80-90)	17197
[40-50)	9685
[30-40)	3775
[90-100)	2793
[20-30)	1657
[10-20)	691
[0-10)	161

```
----------------------------------------------------
Column: weight
----------------------------------------------------
```

weight	count
?	98569
[75-100)	1336
[50-75)	897
[100-125)	625
[125-150)	145
[25-50)	97
[0-25)	48
[150-175)	35
[175-200)	11
>200	3

```
----------------------------------------------------
Column: payer_code
----------------------------------------------------
```

payer_code	count
?	40256
MC	32439
HM	6274
SP	5007
BC	4655
MD	3532
CP	2533
UN	2448
CM	1937
OG	1033

```
----------------------------------------------------
Column: medical_specialty
----------------------------------------------------
```

medical_specialty	count
?	49949
InternalMedicine	14635
Emergency/Trauma	7565
Family/GeneralPractice	7440
Cardiology	5352
Surgery-General	3099
Nephrology	1613
Orthopedics	1400
Orthopedics-Reconstructive	1233
Radiologist	1140

```
----------------------------------------------------
Column: diag_1
----------------------------------------------------
```

diag_1	count
428	6862
414	6581
786	4016
410	3614
486	3508
427	2766
491	2275
715	2151
682	2042
434	2028

```
----------------------------------------------------
Column: diag_2
----------------------------------------------------
```

diag_2	count
276	6752
428	6662
250	6071
427	5036
401	3736
496	3305
599	3288
403	2823
414	2650
411	2566

```
----------------------------------------------------
Column: diag_3
----------------------------------------------------
```

diag_3	count
250	11555
401	8289
276	5175
428	4577
427	3955
414	3664
496	2605
403	2357
585	1992
272	1969

```
----------------------------------------------------
Column: max_glu_serum
----------------------------------------------------
```

max_glu_serum	count
NaN	96420
Norm	2597
>200	1485
>300	1264

```
----------------------------------------------------
Column: A1Cresult
----------------------------------------------------
```

A1Cresult	count
NaN	84748
>8	8216
Norm	4990
>7	3812

```
----------------------------------------------------
Column: metformin
----------------------------------------------------
```

metformin	count
No	81778
Steady	18346
Up	1067
Down	575

```
----------------------------------------------------
Column: repaglinide
----------------------------------------------------
```

repaglinide	count
No	100227
Steady	1384
Up	110
Down	45

```
----------------------------------------------------
Column: nateglinide
----------------------------------------------------
```

nateglinide	count
No	101063
Steady	668
Up	24
Down	11

```
----------------------------------------------------
Column: chlorpropamide
----------------------------------------------------
```

chlorpropamide	count
No	101680
Steady	79
Up	6

Down	1

```
--------------------------------------------------
Column: glimepiride
--------------------------------------------------
```

glimepiride	count
No	96575
Steady	4670
Up	327
Down	194

```
--------------------------------------------------
Column: acetohexamide
--------------------------------------------------
```

acetohexamide	count
No	101765
Steady	1

```
--------------------------------------------------
Column: glipizide
--------------------------------------------------
```

glipizide	count
No	89080
Steady	11356
Up	770
Down	560

```
--------------------------------------------------
Column: glyburide
--------------------------------------------------
```

glyburide	count
No	91116
Steady	9274
Up	812
Down	564

```
--------------------------------------------------
Column: tolbutamide
--------------------------------------------------
```

tolbutamide	count
No	101743
Steady	23

```
--------------------------------------------------
Column: pioglitazone
--------------------------------------------------
```

pioglitazone	count
No	94438
Steady	6976
Up	234
Down	118

```
--------------------------------------------------
Column: rosiglitazone
--------------------------------------------------
```

rosiglitazone	count
No	95401
Steady	6100
Up	178
Down	87

```
--------------------------------------------------
Column: acarbose
--------------------------------------------------
```

acarbose	count
No	101458
Steady	295
Up	10
Down	3

```
--------------------------------------------------
Column: miglitol
--------------------------------------------------
```

miglitol	count
No	101728
Steady	31
Down	5
Up	2

```
--------------------------------------------------
Column: troglitazone
--------------------------------------------------
```

troglitazone	count
No	101763
Steady	3

```
--------------------------------------------------
Column: tolazamide
--------------------------------------------------
```

tolazamide	count
No	101727
Steady	38
Up	1

```
--------------------------------------------------
Column: examide
--------------------------------------------------
```

examide	count
No	101766

```
--------------------------------------------------
Column: citoglipton
--------------------------------------------------
```

citoglipton	count
No	101766

```
--------------------------------------------------
Column: insulin
--------------------------------------------------
```

insulin	count
No	47383
Steady	30849
Down	12218
Up	11316

```
--------------------------------------------------
Column: glyburide-metformin
--------------------------------------------------
```

glyburide-metformin	count
No	101060
Steady	692
Up	8
Down	6

```
--------------------------------------------------
Column: glipizide-metformin
--------------------------------------------------
```

glipizide-metformin	count
No	101753
Steady	13

```
--------------------------------------------------
Column: glimepiride-pioglitazone
--------------------------------------------------
```

glimepiride-pioglitazone	count
No	101765
Steady	1

```
--------------------------------------------------
Column: metformin-rosiglitazone
--------------------------------------------------
```

metformin-rosiglitazone	count
No	101764
Steady	2

```
--------------------------------------------------
Column: metformin-pioglitazone
--------------------------------------------------
```

metformin-pioglitazone	count
No	101765
Steady	1

```
--------------------------------------------------
Column: change
--------------------------------------------------
```

change	count

Next steps: Generate code with one_value_cols | New interactive sheet | Generate code with numeric_summary | New interactive sheet | Generate code with categorical_summary | New interactive sheet | Generate code with flags | New interactive sheet

4) Data Cleaning and Target Definition

Based on the exploratory data analysis, several data quality issues were identified:

- some categorical columns use "?" as a placeholder for missing values
- some columns are identifiers and do not provide predictive value
- some features have a very high proportion of missing values
- the original target variable `readmitted` must be converted into a binary outcome for 30-day readmission prediction

To address these issues, we:

1. replace "?" with standard missing values (`NaN`)
2. remove identifier columns
3. remove high-missing columns
4. define the binary target:
   - 1 if `readmitted == "<30"`
   - 0 otherwise

```
# ------------------------------
# 4.1 Remove obvious non-useful columns
# ------------------------------
cols_to_drop = ["encounter_id", "patient_nbr", "weight", "payer_code", "medical_specialty"]

existing_cols_to_drop = [col for col in cols_to_drop if col in df.columns]
df = df.drop(columns=existing_cols_to_drop)

print("\nRemoved columns:")
print(existing_cols_to_drop)

# ------------------------------
# 4.2 Replace hidden missing values
# ------------------------------
df = df.replace("?", np.nan)

print("\nMissing values after replacing '?' with NaN (top 15):")
missing_after_replace = df.isnull().sum().sort_values(ascending=False)
display(missing_after_replace[missing_after_replace > 0].head(15).to_frame("missing_count"))

# ------------------------------
# 4.3 Define binary target
# ------------------------------
TARGET_COL = "readmitted"

# keep only rows with known target
df = df.dropna(subset=[TARGET_COL]).copy()

# create binary target
df["target_readmit_30"] = (df[TARGET_COL] == "<30").astype(int)

# optional: drop original target to avoid leakage later
df = df.drop(columns=[TARGET_COL])

# ------------------------------
# 4.4 Final check after cleaning
# ------------------------------
print("\nDataset shape after cleaning:", df.shape)

print("\nTarget distribution (count):")
display(df["target_readmit_30"].value_counts().sort_index().to_frame("count"))

print("\nTarget distribution (ratio):")
display(
    df["target_readmit_30"]
    .value_counts(normalize=True)
    .sort_index()
    .round(4)
    .to_frame("ratio")
)

print("\nRemaining missing values (top 15):")
remaining_missing = df.isnull().sum().sort_values(ascending=False)
display(remaining_missing[remaining_missing > 0].head(15).to_frame("missing_count"))

print("\nPreview of cleaned dataset:")
display(df.head())
```

```
Removed columns:
['encounter_id', 'patient_nbr', 'weight', 'payer_code', 'medical_specialty']
Missing values after replacing '?' with NaN (top 15):
```

	missing_count
max_glu_serum	96420
A1Cresult	84748
race	2273
diag_3	1423
diag_2	358
diag_1	21

```
Dataset shape after cleaning: (101766, 45)
Target distribution (count):
```

	count
0	90409
1	11357

```
Target distribution (ratio):
```

	ratio
0	0.8884
1	0.1116

```
Remaining missing values (top 15):
```

	missing_count
max_glu_serum	96420
A1Cresult	84748
race	2273
diag_3	1423
diag_2	358
diag_1	21

```
Preview of cleaned dataset:
```

	race	gender	age	admission_type_id	discharge_disposition_id	admission_source_id	time_in_hospital	num_lab_procedures	num_procedures	num_medications	number_outpatient	number_emergency	number_inpatient	diag_1	diag_2	diag_3	number_diagnoses	max_glu_serum	A1Cresult	metformin	repaglinide	nateglinide	chlorpropamide	glimepiride	acetohexamide	glipizide	glyburi
0	Caucasian	Female	[0-10)	6	25	1	1	41	0	1	0	0	0	250.83	NaN	NaN	1	NaN	NaN	No	No	No	No	No	No	No	N
1	Caucasian	Female	[10-20)	1	1	7	3	59	0	18	0	0	0	276	250.01	255	9	NaN	NaN	No	No	No	No	No	No	No	N
2	AfricanAmerican	Female	[20-30)	1	1	7	2	11	5	13	2	0	1	648	250	V27	6	NaN	NaN	No	No	No	No	No	No	Steady	N
3	Caucasian	Male	[30-40)	1	1	7	2	44	1	16	0	0	0	8	250.43	403	7	NaN	NaN	No	No	No	No	No	No	No	N
4	Caucasian	Male	[40-50)	1	1	7	1	51	0	8	0	0	0	197	157	250	5	NaN	NaN	No	No	No	No	No	No	Steady	N

5) Feature Selection

After cleaning the dataset, we define a subset of features to be used for model training.

The selected variables include:

- healthcare utilization features (e.g., number of visits, medications, procedures)
- demographic variables (age, gender, race)
- diagnosis-related variables (diag_1, diag_2, diag_3)
- admission and discharge information
- diabetes treatment indicators (insulin, diabetesMed, change)

This subset keeps clinically relevant information while avoiding unnecessary variables that may introduce noise or leakage.

```
candidate_cols = [
    "time_in_hospital",
    "num_lab_procedures",
    "num_procedures",
    "num_medications",
    "number_outpatient",
    "number_emergency",
    "number_inpatient",
    "race",
    "gender",
    "age",
    "admission_type_id",
    "discharge_disposition_id",
    "admission_source_id",
    "insulin",
    "change",
    "diabetesMed"
]

# Check which columns exist
available_cols = [c for c in candidate_cols if c in df.columns]
missing_cols = [c for c in candidate_cols if c not in df.columns]

print("Number of selected features:", len(available_cols))
print("Missing columns:", missing_cols)

# Define features and target
X = df[available_cols].copy()
y = df["target_readmit_30"].copy()

print("\nFeature matrix shape:", X.shape)
print("Target shape:", y.shape)

display(X.head())
```

```
Number of selected features: 16
Missing columns: []

Feature matrix shape: (101766, 16)
Target shape: (101766,)
```

	time_in_hospital	num_lab_procedures	num_procedures	num_medications	number_outpatient	number_emergency	number_inpatient	race	gender	age	admission_type_id	discharge_disposition_id	admission_source_id	insulin	change	diabetesMed
0	1	41	0	1	0	0	0	Caucasian	Female	[0-10)	6	25	1	No	No	No
1	3	59	0	18	0	0	0	Caucasian	Female	[10-20)	1	1	7	Up	Ch	Yes
2	2	11	5	13	2	0	1	AfricanAmerican	Female	[20-30)	1	1	7	No	No	Yes
3	2	44	1	16	0	0	0	Caucasian	Male	[30-40)	1	1	7	Up	Ch	Yes
4	1	51	0	8	0	0	0	Caucasian	Male	[40-50)	1	1	7	Steady	Ch	Yes

6) Exploratory Data Analysis (EDA)

This section explores relationships between selected features and the binary target variable.

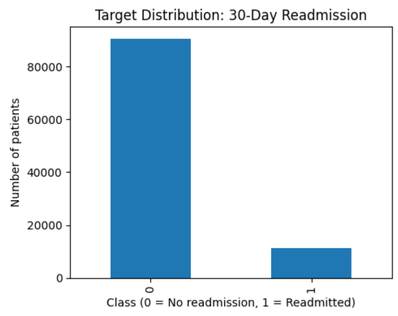
The goal is to better understand the dataset before training machine learning models and to identify potential predictors of 30-day hospital readmission.

We visualize:

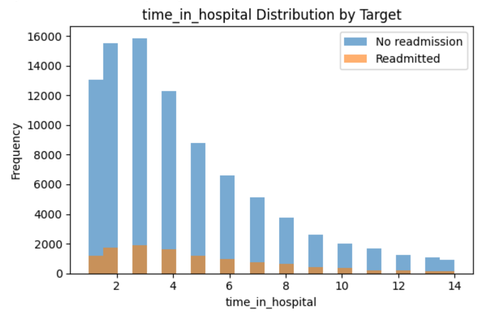
- the distribution of the target variable
- key healthcare utilization variables by target class
- readmission patterns by age group
- readmission patterns by insulin treatment status

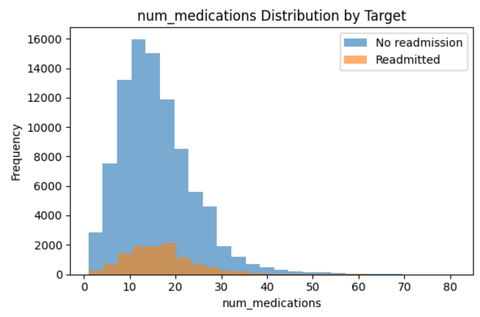
```
# ----------------------------------------------------------
# 6.1 Target distribution
# ----------------------------------------------------------
print("\nTarget distribution")

fig, ax = plt.subplots(figsize=(5, 4))
y.value_counts().sort_index().plot(kind="bar", ax=ax)

ax.set_title("Target Distribution: 30-Day Readmission")
ax.set_xlabel("Class (0 = No readmission, 1 = Readmitted)")
ax.set_ylabel("Number of patients")

plt.tight_layout()
plt.show()

# ----------------------------------------------------------
# 6.2 Key numeric variables by target
# ----------------------------------------------------------
print("\nKey numeric variables by target")

numeric_cols = ["time_in_hospital", "num_medications", "number_inpatient"]

for col in numeric_cols:
    if col in X.columns:
        fig, ax = plt.subplots(figsize=(6, 4))

        X.loc[y == 0, col].dropna().plot(
            kind="hist",
            bins=25,
            alpha=0.6,
            label="No readmission",
            ax=ax
        )

        X.loc[y == 1, col].dropna().plot(
            kind="hist",
            bins=25,
            alpha=0.6,
            label="Readmitted",
            ax=ax
        )

        ax.set_title(f"{col} Distribution by Target")
        ax.set_xlabel(col)
        ax.set_ylabel("Frequency")
        ax.legend()

        plt.tight_layout()
        plt.show()

# ----------------------------------------------------------
# 6.3 Readmission rate by age group
# ----------------------------------------------------------
if "age" in X.columns:
    print("\nReadmission rate by age group")

    age_readmit = (
        pd.DataFrame({"age": X["age"], "target": y})
        .groupby("age")["target"]
        .mean()
        .sort_index()
    )

    fig, ax = plt.subplots(figsize=(8, 4))
    age_readmit.plot(kind="bar", ax=ax)

    ax.set_title("Average 30-Day Readmission Rate by Age Group")
    ax.set_ylabel("Readmission rate")
    ax.set_xlabel("Age group")

    plt.tight_layout()
    plt.show()

# ----------------------------------------------------------
# 6.4 Readmission rate by insulin treatment
# ----------------------------------------------------------
if "insulin" in X.columns:
    print("\nReadmission rate by insulin treatment")

    insulin_readmit = (
        pd.DataFrame({"insulin": X["insulin"], "target": y})
        .dropna()
        .groupby("insulin")["target"]
        .mean()
        .sort_values(ascending=False)
    )

    fig, ax = plt.subplots(figsize=(6, 4))
    insulin_readmit.plot(kind="bar", ax=ax)

    ax.set_title("Average Readmission Rate by Insulin Treatment")
    ax.set_ylabel("Readmission rate")
    ax.set_xlabel("Insulin category")

    plt.tight_layout()
    plt.show()
```

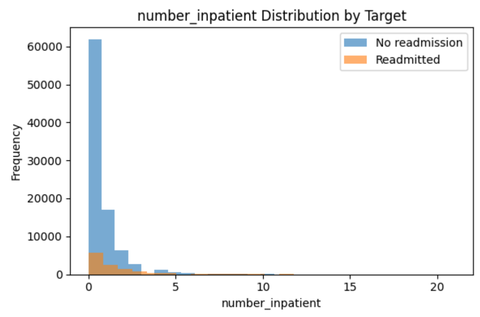
```
Target distribution
```



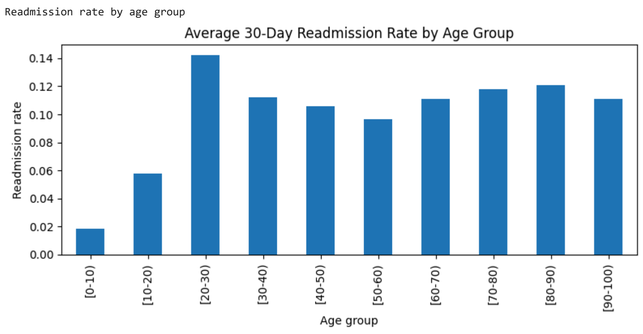

```
Key numeric variables by target
```



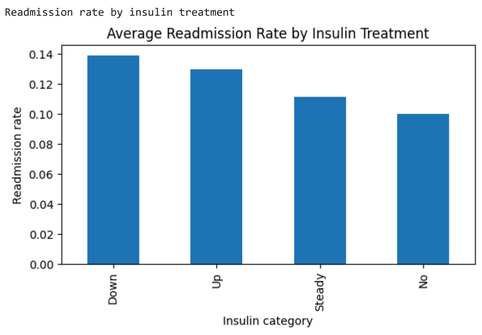





```
Readmission rate by age group
```




```
Readmission rate by insulin treatment
```



7) Stratified Train/Test Split

To properly evaluate the machine learning models, the dataset is divided into a training set and a testing set.

Because the positive class (30-day readmission) is less frequent, we use a **stratified split** to preserve the class distribution in both subsets.

This ensures that the proportion of readmitted patients remains similar in the training and test datasets, which helps produce a more reliable evaluation of model performance.

```
# Split dataset while preserving class distribution
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=TEST_SIZE,
    random_state=RANDOM_STATE,
    stratify=y
)

# Display dataset shapes
print("\nDataset shapes")
print("Train shape:", X_train.shape)
print("Test shape :", X_test.shape)

# Check class distribution
print("\nClass distribution check")

print("Train positive rate:", round(y_train.mean(), 4))
print("Test positive rate :", round(y_test.mean(), 4))
```

```
Dataset shapes
Train shape: (81412, 16)
Test shape : (20354, 16)

Class distribution check
Train positive rate: 0.1116
Test positive rate : 0.1116
```

8) Build the preprocessing pipeline

Before training machine learning models, the dataset must be properly preprocessed.

We build a preprocessing pipeline that automatically handles numeric and categorical features.

The pipeline performs the following operations:

• **Numeric features**

- Missing values are filled using median imputation
- Features are standardized using scaling

• **Categorical features**

- Missing values are filled using the most frequent category
- Categories are converted to numeric format using one-hot encoding

Using a pipeline ensures that preprocessing steps are applied consistently to both the training and test datasets.
This also helps prevent data leakage and makes the workflow fully reproducible.

```
# Identify numeric and categorical columns
numeric_features = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_features = [c for c in X.columns if c not in numeric_features]

print("Numeric features:", numeric_features)
print("Categorical features:", categorical_features)

# -------------------------------------------------------
# Numeric preprocessing
# -------------------------------------------------------
numeric_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler())
    ]
)

# -------------------------------------------------------
# Categorical preprocessing
# -------------------------------------------------------
try:
    onehot = OneHotEncoder(handle_unknown="ignore", sparse_output=False)
except TypeError:
    # Compatibility with older scikit-learn versions
    onehot = OneHotEncoder(handle_unknown="ignore", sparse=False)

categorical_transformer = Pipeline(
    steps=[
        ("imputer", SimpleImputer(strategy="most_frequent")),
        ("onehot", onehot)
    ]
)

# -------------------------------------------------------
# Combine preprocessing steps
# -------------------------------------------------------
```

```
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

print("\nPreprocessing pipeline successfully created.")
```

```
Numeric features: ['time_in_hospital', 'num_lab_procedures', 'num_procedures', 'num_medications', 'number_outpatient', 'number_emergency', 'number_inpatient', 'admission_type_id', 'discharge_disposition_id', 'admission_source_id']
Categorical features: ['race', 'gender', 'age', 'insulin', 'change', 'diabetesMed']

Preprocessing pipeline successfully created.
```

9) Compare Classification Models with 5-Fold Cross-Validation

In this section, we compare several classification algorithms using 5-fold stratified cross-validation on the training set.

The models evaluated are:

- Logistic Regression
- Random Forest
- Gradient Boosting
- Neural Network (MLPClassifier)

The following performance metrics are reported:

- Accuracy
- Precision
- Recall
- F1-score
- ROC-AUC

For this healthcare prediction task, **recall** and **F1-score** are especially important because failing to identify high-risk patients may have important clinical consequences.

```
# ----------------------------------------------------------
# Candidate models
# ----------------------------------------------------------
candidate_models = {
    "Logistic Regression": LogisticRegression(
        max_iter=3000,
        class_weight="balanced",
        random_state=RANDOM_STATE
    ),
    "Random Forest": RandomForestClassifier(
        n_estimators=300,
        class_weight="balanced",
        random_state=RANDOM_STATE,
        n_jobs=-1
    ),
    "Gradient Boosting": GradientBoostingClassifier(
        random_state=RANDOM_STATE
    ),
    "MLP Neural Network": MLPClassifier(
        hidden_layer_sizes=(64, 32),
        activation="relu",
        max_iter=300,
        early_stopping=True,
        random_state=RANDOM_STATE
    )
}

# ----------------------------------------------------------
# Cross-validation strategy
# ----------------------------------------------------------
cv = StratifiedKFold(
    n_splits=CV_FOLDS,
    shuffle=True,
    random_state=RANDOM_STATE
)

# ----------------------------------------------------------
# Evaluation metrics
# ----------------------------------------------------------
scoring = {
    "accuracy": "accuracy",
    "precision": "precision",
    "recall": "recall",
    "f1": "f1",
    "roc_auc": "roc_auc"
}

# ----------------------------------------------------------
# Model comparison
# ----------------------------------------------------------
comparison_results = []

for model_name, model in candidate_models.items():

    pipeline = Pipeline(
        steps=[
            ("preprocess", preprocessor),
            ("model", model)
        ]
    )

    scores = cross_validate(
        estimator=pipeline,
        X=X_train,
        y=y_train,
        cv=cv,
        scoring=scoring,
        n_jobs=-1,
        error_score="raise"
    )

    comparison_results.append({
        "model": model_name,
        "cv_accuracy_mean": scores["test_accuracy"].mean(),
        "cv_precision_mean": scores["test_precision"].mean(),
        "cv_recall_mean": scores["test_recall"].mean(),
        "cv_f1_mean": scores["test_f1"].mean(),
        "cv_roc_auc_mean": scores["test_roc_auc"].mean()
    })

# ----------------------------------------------------------
# Results table
# ----------------------------------------------------------
results_df = pd.DataFrame(comparison_results)

results_df = results_df.sort_values(
    by=["cv_f1_mean", "cv_recall_mean", "cv_roc_auc_mean"],
    ascending=False
).reset_index(drop=True)

results_df = results_df.round(4)

display(results_df)
```

	model	cv_accuracy_mean	cv_precision_mean	cv_recall_mean	cv_f1_mean	cv_roc_auc_mean
0	Logistic Regression	0.6740	0.1705	0.4970	0.2539	0.6380
1	MLP Neural Network	0.8885	0.5321	0.0110	0.0216	0.6402
2	Gradient Boosting	0.8885	0.5375	0.0090	0.0177	0.6658
3	Random Forest	0.8879	0.3805	0.0067	0.0132	0.6253

Next steps: Generate code with `results_df` New interactive sheet

10) Hyperparameter Tuning with RandomizedSearchCV

After comparing the candidate models, we tune the best-performing model using `RandomizedSearchCV`.

This approach is more computationally efficient than an exhaustive grid search while still exploring multiple hyperparameter combinations.

The tuning process is based on **F1-score**, which is appropriate for this healthcare classification task because it balances precision and recall.

```
# ----------------------------------------------------------
# Select best model from section 9
# ----------------------------------------------------------
best_model_name = results_df.loc[0, "model"]
print("Best model selected for tuning:", best_model_name)

# ----------------------------------------------------------
# Define model and parameter space
# ----------------------------------------------------------
if best_model_name == "Random Forest":
    base_model = RandomForestClassifier(
        class_weight="balanced",
        random_state=RANDOM_STATE,
        n_jobs=-1
    )
    param_distributions = {
        "model__n_estimators": [100, 150, 200],
        "model__max_depth": [None, 8, 12, 20],
        "model__min_samples_split": [2, 5, 10],
        "model__min_samples_leaf": [1, 2, 4]
    }

elif best_model_name == "Logistic Regression":
    base_model = LogisticRegression(
        max_iter=5000,
        class_weight="balanced",
        random_state=RANDOM_STATE
    )
    param_distributions = {
        "model__C": np.logspace(-2, 2, 8),
        "model__solver": ["lbfgs", "liblinear"]
    }

elif best_model_name == "Gradient Boosting":
    base_model = GradientBoostingClassifier(
        random_state=RANDOM_STATE
    )
    param_distributions = {
        "model__n_estimators": [100, 150, 200],
        "model__learning_rate": [0.03, 0.05, 0.1, 0.2],
        "model__max_depth": [2, 3, 4]
    }

else:
    raise ValueError("Unexpected best model name. Please check results_df.")

# ----------------------------------------------------------
# Build tuning pipeline
# ----------------------------------------------------------
tuning_pipeline = Pipeline(
    steps=[
        ("preprocess", preprocessor),
        ("model", base_model)
    ]
)

# ----------------------------------------------------------
# Randomized search
# ----------------------------------------------------------
random_search = RandomizedSearchCV(
    estimator=tuning_pipeline,
    param_distributions=param_distributions,
    n_iter=5,              # reduced for faster execution
    scoring="f1",
    cv=CV_FOLDS,            # use 3-fold CV if defined earlier
    random_state=RANDOM_STATE,
    n_jobs=-1,
    verbose=1
)

random_search.fit(X_train, y_train)

# ----------------------------------------------------------
# Best tuning results
# ----------------------------------------------------------
```

```
print("\nBest parameters:")
display(random_search.best_params_)

print("Best cross-validated F1-score:", round(random_search.best_score_, 4))

# Save best tuned model
best_model = random_search.best_estimator_
```

```
Best model selected for tuning: Logistic Regression
Fitting 5 folds for each of 5 candidates, totalling 25 fits

Best parameters:
{'model__solver': 'lbfgs', 'model__C': np.float64(0.01)}
Best cross-validated F1-score: 0.253
```

11) Final Evaluation on the Held-Out Test Set

After selecting and tuning the best model, we evaluate its performance on the unseen test set.

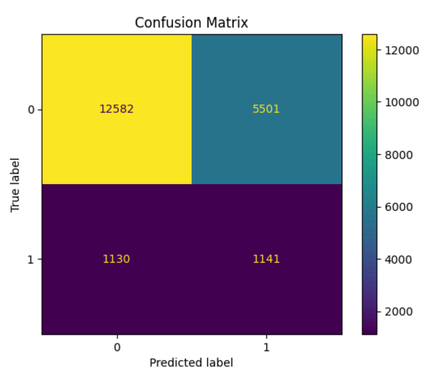
The following outputs are reported:

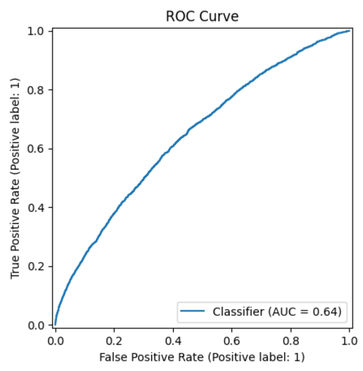
- test-set performance metrics
- classification report
- confusion matrix
- ROC-AUC
- ROC curve

This final evaluation provides an estimate of how well the tuned model generalizes to new patient records.

```
# ----------------------------------------------------------
# Best tuned pipeline
# ----------------------------------------------------------
best_pipe = random_search.best_estimator_

# ----------------------------------------------------------
# Predictions
# ----------------------------------------------------------
y_pred = best_pipe.predict(X_test)
y_proba = None

if hasattr(best_pipe.named_steps["model"], "predict_proba"):
    y_proba = best_pipe.predict_proba(X_test)[:, 1]

elif hasattr(best_pipe.named_steps["model"], "decision_function"):
    decision_values = best_pipe.decision_function(X_test)

    # Normalize values to [0,1] for ROC-AUC / ROC curve visualization
    y_proba = (
        (decision_values - decision_values.min()) /
        (decision_values.max() - decision_values.min() + 1e-12)
    )

# ----------------------------------------------------------
# Test metrics
# ----------------------------------------------------------
test_metrics = {
    "accuracy": accuracy_score(y_test, y_pred),
    "precision": precision_score(y_test, y_pred, zero_division=0),
    "recall": recall_score(y_test, y_pred, zero_division=0),
    "f1": f1_score(y_test, y_pred, zero_division=0)
}

if y_proba is not None:
    test_metrics["roc_auc"] = roc_auc_score(y_test, y_proba)

print("\nTest metrics:")
for metric_name, metric_value in test_metrics.items():
    print(f"{metric_name}: {metric_value:.4f}")

# ----------------------------------------------------------
# Classification report
# ----------------------------------------------------------
print("\nClassification report:")
print(classification_report(y_test, y_pred, digits=4))

# ----------------------------------------------------------
# Confusion matrix
# ----------------------------------------------------------
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(confusion_matrix=cm)
disp.plot()
plt.title("Confusion Matrix")
plt.show()

# ----------------------------------------------------------
# ROC curve
# ----------------------------------------------------------
if y_proba is not None:
    RocCurveDisplay.from_predictions(y_test, y_proba)
    plt.title("ROC Curve")
    plt.show()
```

```
Test metrics:
accuracy: 0.6742
precision: 0.1718
recall: 0.5024
f1: 0.2560
roc_auc: 0.6449

Classification report:
              precision    recall  f1-score   support

           0     0.9176    0.6958    0.7914     18083
           1     0.1718    0.5024    0.2560      2271

    accuracy                         0.6742     20354
   macro avg     0.5447    0.5991    0.5237     20354
weighted avg     0.8344    0.6742    0.7317     20354
```





12) Quick Interpretation of Model Drivers

To better understand what influences the model predictions, we inspect the most important features used by the trained model.

Depending on the selected algorithm:

- Tree-based models (Random Forest, Gradient Boosting) provide **feature importance scores**
- Linear models (Logistic Regression) provide **coefficients**

Examining the most influential variables helps connect the machine learning results back to the healthcare problem and identify which patient characteristics are most associated with hospital readmission.

```
selected_model = best_pipe.named_steps["model"]

# ----------------------------------------------------------
# Recover feature names after preprocessing
# ----------------------------------------------------------
cat_names = []

if len(categorical_features) > 0:
    onehot_fitted = (
        best_pipe
        .named_steps["preprocess"]
        .named_transformers_["cat"]
        .named_steps["onehot"]
    )
    cat_names = list(onehot_fitted.get_feature_names_out(categorical_features))

feature_names = numeric_features + cat_names

top_features_df = pd.DataFrame()

# ----------------------------------------------------------
# Tree-based models (feature importance)
# ----------------------------------------------------------
if hasattr(selected_model, "feature_importances_"):

    top_features_df = (
        pd.DataFrame({
            "feature": feature_names,
            "importance": selected_model.feature_importances_
        })
        .sort_values("importance", ascending=False)
        .head(20)
    )

    print("\nTop important features:")
    display(top_features_df)

# ----------------------------------------------------------
# Linear models (coefficients)
# ----------------------------------------------------------
elif hasattr(selected_model, "coef_"):

    coef_values = np.ravel(selected_model.coef_)

    top_features_df = (
        pd.DataFrame({
            "feature": feature_names,
            "coefficient": coef_values,
            "abs_coefficient": np.abs(coef_values)
        })
        .sort_values("abs_coefficient", ascending=False)
        .head(20)
    )

    print("\nTop influential coefficients:")
    display(top_features_df[["feature", "coefficient"]])
```

```
# ----------------------------------------------------------
# Other models
# ----------------------------------------------------------
else:
    print("This model does not provide simple built-in feature importance.")
```

```
Top influential coefficients:
```

	feature	coefficient
6	number_inpatient	0.370059
19	age_[10-20)	-0.286732
26	age_[80-90)	0.162790
25	age_[70-80)	0.143779
28	insulin_Down	0.139093
8	discharge_disposition_id	0.135591
18	age_[0-10)	-0.129361
29	insulin_No	-0.126785
34	diabetesMed_No	-0.086664
13	race_Hispanic	-0.075189
32	change_Ch	-0.068860
0	time_in_hospital	0.068429
30	insulin_Steady	-0.065263
24	age_[60-70)	0.064388
5	number_emergency	0.063708
27	age_[90-100)	0.063207
23	age_[50-60)	-0.061171
3	num_medications	0.046634
35	diabetesMed_Yes	0.044695
15	gender_Female	-0.034309

13) Patient-Level Prediction Example

To make the project more concrete, we generate a prediction for a single patient from the test dataset.

This example illustrates how the trained machine learning pipeline can be applied to an individual patient record. The model predicts whether the patient is likely to be readmitted within 30 days.

The output includes:

- the true label from the dataset
- the predicted label from the model
- the predicted probability of readmission
- the patient's structured feature values

This patient-level example will later be used to generate a **grounded Generative AI summary** in the next section.

```
# ----------------------------------------------------------
# Select a patient from the test set
# ----------------------------------------------------------

patient_index = X_test.index[0]
patient_features = X_test.loc[[patient_index]].copy()
patient_true_label = y_test.loc[patient_index]

# ----------------------------------------------------------
# Generate prediction
# ----------------------------------------------------------

patient_pred = int(best_pipe.predict(patient_features)[0])

# ----------------------------------------------------------
# Generate probability score
# ----------------------------------------------------------

if hasattr(best_pipe.named_steps["model"], "predict_proba"):

    patient_risk_score = float(
        best_pipe.predict_proba(patient_features)[0, 1]
    )

elif hasattr(best_pipe.named_steps["model"], "decision_function"):

    val = float(
        best_pipe.named_steps["model"].decision_function(
            best_pipe.named_steps["preprocess"].transform(patient_features)
        )[0]
    )

    # convert decision score to probability-like value
    patient_risk_score = float(1 / (1 + np.exp(-val)))

else:

    patient_risk_score = float("nan")

# ----------------------------------------------------------
# Display results
# ----------------------------------------------------------

print("Patient index:", patient_index)
print("True label:", int(patient_true_label))
print("Predicted label:", patient_pred)
print("Predicted 30-day readmission probability:", round(patient_risk_score, 4))

print("\nPatient feature snapshot:")
display(patient_features)
```

```
Patient index: 99195
True label: 0
Predicted label: 1
Predicted 30-day readmission probability: 0.528

Patient feature snapshot:
```

	time_in_hospital	num_lab_procedures	num_procedures	num_medications	number_outpatient	number_emergency	number_inpatient	race	gender	age	admission_type_id	discharge_disposition_id	admission_source_id	insulin	change	diabetesMed
99195	4	40	0	17	0	0	1	Caucasian	Female	[80-90)	1	1	7	Up	Ch	Yes

14) Generative AI Integration — Grounded Patient Summary

In this final step, Generative AI is used only for **communication and summarization**.

The predictive model itself is produced by the classical machine learning pipeline developed in the previous sections. The generative model does **not influence the prediction**.

Instead, it receives:

- the patient's structured features
- the model prediction
- the predicted probability
- the most important model drivers

Using this information, the generative model produces a **short, human-readable explanation of the prediction**.

To ensure responsible use of generative AI, the prompt follows three rules:

• The explanation must rely only on the provided data
• The model must not invent missing medical information
• The output must clearly state that it is a model-generated summary for educational purposes

This approach demonstrates a safe and transparent way to integrate Generative AI into a healthcare analytics workflow.

```
# ==========================================================
# 14. GENERATIVE AI INTEGRATION — GROUNDED PATIENT SUMMARY
# ==========================================================

def safe_patient_snapshot(row: pd.DataFrame, preferred_cols=None):

    preferred_cols = preferred_cols or [
        "age",
        "time_in_hospital",
        "num_medications",
        "number_inpatient",
        "number_emergency",
        "number_outpatient",
        "insulin",
        "change",
        "diabetesMed"
    ]

    available = [c for c in preferred_cols if c in row.columns]

    return row[available].iloc[0].to_dict()


def get_top_driver_list(top_df: pd.DataFrame, n=5):

    if top_df.empty:
        return ["Top drivers were not available for this model."]

    if "importance" in top_df.columns:
        return [
            f"{r.feature} (importance={r.importance:.4f})"
            for r in top_df.head(n).itertuples()
        ]

    if "coefficient" in top_df.columns:
        return [
            f"{r.feature} (coefficient={r.coefficient:.4f})"
            for r in top_df.head(n).itertuples()
        ]

    return top_df.head(n).astype(str).agg(" | ".join, axis=1).tolist()


# ----------------------------------------------------------
# Extract patient information
# ----------------------------------------------------------

patient_snapshot = safe_patient_snapshot(patient_features)
driver_lines = get_top_driver_list(top_features_df, n=5)

risk_label = "high risk" if patient_pred == 1 else "lower risk"


# ----------------------------------------------------------
# Build grounded prompt for GenAI
# ----------------------------------------------------------

patient_prompt = f"""
You are a healthcare data analyst.

Task:
Write a short, careful patient-level summary based ONLY on the structured data below and the model output.

Rules:
- Do NOT invent symptoms, diagnoses, or treatments that are not provided.
- Do NOT give medical advice.
- State clearly that this is a model-generated risk summary for educational purposes only.
- Mention uncertainty when appropriate.

Output format:
1) Prediction
2) Supporting factors
3) Important limitations

Patient structured data:
{patient_snapshot}

Model output:
- Predicted class: {patient_pred}
  (1 = readmitted within 30 days, 0 = not readmitted within 30 days)
```

```
- Predicted probability of 30-day readmission:
  {patient_risk_score:.4f}

- Risk interpretation:
  {risk_label}

Top model drivers:
{driver_lines}
"""

print(patient_prompt)
```

```
You are a healthcare data analyst.

Task:
Write a short, careful patient-level summary based ONLY on the structured data below and the model output.

Rules:
- Do NOT invent symptoms, diagnoses, or treatments that are not provided.
- Do NOT give medical advice.
- State clearly that this is a model-generated risk summary for educational purposes only.
- Mention uncertainty when appropriate.

Output format:
1) Prediction
2) Supporting factors
3) Important limitations

Patient structured data:
{'age': '[80-90)', 'time_in_hospital': 4, 'num_medications': 17, 'number_inpatient': 1, 'number_emergency': 0, 'number_outpatient': 0, 'insulin': 'Up', 'change': 'Ch', 'diabetesMed': 'Yes'}

Model output:
- Predicted class: 1
  (1 = readmitted within 30 days, 0 = not readmitted within 30 days)

- Predicted probability of 30-day readmission:
  0.5280

- Risk interpretation:
  high risk

Top model drivers:
['number_inpatient (coefficient=0.3701)', 'age_[10-20) (coefficient=-0.2867)', 'age_[80-90) (coefficient=0.1628)', 'age_[70-80) (coefficient=0.1438)', 'insulin_Down (coefficient=0.1391)']
```

Optional LLM Call

If a Large Language Model (LLM) API is available, the grounded prompt created above can be sent to a generative model to produce a short patient summary.

In this project, the generative model is used only for **communication and explanation**, while the prediction itself is produced by the classical machine learning pipeline.

The prompt is grounded in:

- the patient's structured features
- the model prediction
- the predicted probability
- the most important model drivers

The API call is kept **optional and commented** so that the notebook remains fully reproducible without requiring external API access.

```
# ============================================================
# OPTIONAL LLM CALL (example only)
# ============================================================
# This shows how the grounded prompt could be sent to a
# generative model if an API key is available.
# The code is intentionally commented so the notebook runs
# without requiring external API access.

# from openai import OpenAI
# client = OpenAI()

# response = client.responses.create(
#     model="gpt-4.1-mini",
#     input=patient_prompt
# )

# print(response.output_text)
```

15) Executive Summary Prompt for the Overall Model

This section prepares a structured prompt that can be used by a Generative AI model to produce an executive-level summary of the project.